Développer une Application Web d'Analyse de Sentiment Basée sur une API

Développer une Application Web d'Analyse de Sentiment Basée sur une API 0
Contraintes générales pour tous les projets : 0
Installation des Bibliothèques et Préparation du Modèle : 1
Bibliothèques : 1
Énoncé : 2
Actions à mener 2
Architecture de l'application : 3
L'analyseur de sentiment : 4
Analyser un sentiment 4
Validation des données 5
Journalisation 6
Retourner un dictionnaire valide : 6
Envoyer une requête à l'API et afficher les résultats dans streamlit 7
Gérer les résultats 7
Gérer les exceptions : 8
du côté streamlit 8
du côté Fast API 8

Contraintes générales pour tous les projets :

- Un environnement virtuel (`.venv`) pour isoler les dépendances.
- Un fichier `requirements.txt` doit lister toutes les dépendances du projet.
- Un fichier `README.md` doit décrire le projet, et les instructions d'installation.
- Un fichier `.ignore` doit refuser le versionnage du `.venv`.

Installation des Bibliothèques et Préparation du Modèle :

Création d'un environnement virtuel :

- Ouvrez votre terminal ou invite de commandes.
- Exécutez la commande : `python -m venv .venv`

Activation de l'environnement virtuel :

- Sous Windows : `.venv\Scripts\activate`
- Sous macOS/Linux : `source .venv/bin/activate`

Installation des dépendances :

- Assurez-vous d'être dans le répertoire du projet.
- Exécutez la commande : `pip install -r requirements.txt`
- Sinon installer les bibliothèques vous même

Téléchargement du lexique VADER :

- Exécutez la commande :
  `python -c "import nltk; nltk.download('vader_lexicon')"`

Ceci télécharge ce dont a besoin le model NLTK VADER pour fonctionner.

Bibliothèques :

- `nltk`
- `fastapi uvicorn`
- `streamlit`
- `requests`
- `pydantic`

- `loguru`

Énoncé :

Ce projet vise à développer une application web fonctionnelle qui intègre un modèle d'intelligence artificielle simple pour l'analyse de sentiment. L'objectif est de créer une architecture modulaire et réutilisable, en séparant la logique de traitement des données de l'interface utilisateur. Cette application permettra aux utilisateurs de saisir du texte et de recevoir une analyse de son sentiment (positif, négatif, neutre).

Actions à mener

Installation de l'environnement :
Suivre les instructions détaillées fournies précédemment pour créer un environnement virtuel Python (`.venv`) et y installer toutes les dépendances nécessaires et un fichier requirements.txt.

Création des fichiers et dossiers :
Créer les fichiers `sentiment_api.py` et `sentiment_streamlit.py` à la racine du projet. Créer un dossier `logs/` pour stocker les fichiers journaux.

Initialisation du dépôt GitHub :
Initialiser un nouveau dépôt sur Github

Test du modèle de manière simple (Anglophone):
Dans un script Python temporaire ou un notebook de développement, importer `SentimentIntensityAnalyzer` de `NLTK`. Créer une instance de l'analyseur et tester l'analyse de sentiment sur un mot ou une courte phrase :

```
from nltk.sentiment import SentimentIntensityAnalyzer
sia = SentimentIntensityAnalyzer()
sentiment = sia.polarity_scores("bonjour")
print(sentiment)
```

Vérifiez que le dictionnaire de scores de polarité est correctement retourné.

Création de l'API FastAPI :
Dans le fichier **`sentiment_api.py`**, implémenter un point de terminaison **`/analyse_sentiment/`** qui reçoit du texte via une requête POST. Utiliser le modèle VADER pour analyser le sentiment du texte reçu. Retourner le dictionnaire contenant les quatre mesures de sentiment (négatif, neutre, positif, composé) au format JSON.

Ajout de la validation de données :
Utiliser **`Pydantic`** pour définir un modèle de données (par exemple, Texte) qui valide le format du texte reçu par l'API.

Ajout de la journalisation (API) :
Intégrer la bibliothèque loguru pour journaliser les requêtes, les résultats de l'analyse et les éventuelles erreurs dans le fichier **`logs/sentiment_api.log`**.

Création de l'interface Streamlit :
Dans le fichier **`sentiment_streamlit.py`**, développer une interface utilisateur avec une zone de texte pour la saisie du texte et un bouton "**`Analyser`**". Utiliser la bibliothèque **`requests`** pour envoyer le texte saisi à l'API FastAPI via une requête POST. Afficher les résultats de l'analyse de sentiment renvoyés par l'API.

Ajout de la journalisation (Streamlit) :
Utiliser loguru pour journaliser les entrées de l'utilisateur, les résultats de l'API et les éventuelles erreurs dans le fichier **`logs/sentiment_streamlit.log`**.

Architecture de l'application :

```
AnalyseSentiment/
├── .venv                       # L'environnement virtuel
├── api.py                      # API FastAPI pour l'analyse de sentiment
├── app.py                      # Application Streamlit pour l'interface utilisateur
├── requirements.txt            # Liste des dépendances Python
├── logs/                       # Dossier pour les logs
│   ├── sentiment_api.log       # Logs de l'API FastAPI
│   └── sentiment_streamlit.log # logs de l'application Streamlit.
└── README.md                   # Documentation du projet
```

L'analyseur de sentiment :

La classe **`SentimentIntensityAnalyzer`** de NLTK, une fois initialisée avec

```
from nltk.sentiment import SentimentIntensityAnalyzer
sia = SentimentIntensityAnalyzer()
```

permet d'analyser le sentiment d'un texte anglais donné. Elle prend en entrée une chaîne de caractères (le texte à analyser) et renvoie un dictionnaire contenant quatre scores de polarité :

- neg (négatif)
- neu (neutre)
- pos (positif)
- compound.

Les scores neg, neu et pos représentent respectivement la proportion de mots du texte considérés comme négatifs, neutres ou positifs, chacun étant un nombre réel entre 0 et 1.

Le score compound, quant à lui, est une métrique normalisée calculée à partir des trois autres scores, qui résume l'intensité du sentiment global du texte, variant de -1 (sentiment extrêmement négatif) à +1 (sentiment extrêmement positif). Ainsi, l'outil prend du texte brut en entrée et produit une mesure quantitative du sentiment exprimé dans ce texte.

Exemple d'un log de sortie pour le mot "congratulation!"

```
2025-04-03 18:21:29.966 | INFO     | __main__:<module>:13 - Texte à
analyser: congratulation!
2025-04-03 18:21:29.977 | INFO     | __main__:<module>:30 - Résultats
affichés: {'neg': 0.0, 'neu': 0.0, 'pos': 1.0, 'compound': 0.636}
```

Analyser un sentiment

La fonction **`sia.polarity_scores()`** est au cœur de l'analyse de sentiment réalisée par l'API. Elle prend en entrée un texte et détermine sa polarité émotionnelle. Dans notre cas spécifique, cette fonction ne reçoit pas directement une simple chaîne de caractères, mais plutôt un objet **`texte`** géré par **`Pydantic`**.

```
sentiment = sia.polarity_scores(texte_object.texte)
```

Validation des données

Pydantic, une bibliothèque Python pour la validation de données et la gestion des paramètres, permet de définir des modèles de données en utilisant des annotations de type Python standard.

```
from pydantic import BaseModel
class Texte(BaseModel):
    texte: str
```

BaseModel sert de classe de base pour la création de ces modèles. La classe **Texte** qui hérite de BaseModel définit un modèle de données simple avec un seul champ : **texte**. L'annotation de type **str** indique que ce champ doit contenir une chaîne de caractères.

Concrètement, cela signifie que lorsque l'API FastAPI reçoit une requête contenant des données qui devraient correspondre au modèle **Texte**, **Pydantic** s'assure que :

- Les données reçues contiennent un champ nommé "texte".
- La valeur de ce champ est bien une chaîne de caractères.

Si les données ne correspondent pas à ces critères, **Pydantic** lève une exception, signalant une erreur de validation. Cela garantit que les données entrantes respectent le format attendu, ce qui est essentiel pour la fiabilité et la sécurité de l'API.

Ainsi, cette approche permet :

- Une validation automatique des données.
- Une documentation claire du format des données attendues (url/docs).
- Une réduction des risques d'erreurs dues à des données mal formées.

Par exemple pour appliquer le modèle Pydantic comme paramètre de la fonction appelée par la route **analyse_sentiment/**

```
@app.post("/analyse_sentiment/")
async def analyse_sentiment(texte_object: Texte):
```

Journalisation

La bibliothèque **`loguru`** est utilisée pour la journalisation (logging) dans l'application, offrant une manière simple et puissante de suivre les événements et les erreurs. La ligne

```
# Configuration de Loguru
logger.add("logs/sentiment_api.log", rotation="500 MB", level="INFO")
```

configure **`loguru`** pour enregistrer les messages dans un fichier nommé "sentiment_api.log" situé dans le dossier "logs/".

La rotation de fichier est activée, de sorte que lorsque le fichier atteint 500 MB, un nouveau fichier de log est créé, évitant ainsi un fichier de log excessivement volumineux. Seuls les messages de niveau "**`INFO`**" et supérieur (comme les erreurs) seront enregistrés.

Ensuite, l'appel

```
sentiment = sia.polarity_scores(texte_object.texte)
logger.info(f"Résultats: {sentiment}")
```

enregistre un message d'information indiquant qu'une analyse de texte est en cours, incluant le texte analysé dans le message. Cela permet de suivre les requêtes reçues par l'API, les données traitées et de faciliter le débogage en fournissant un historique détaillé des opérations.

Retourner un dictionnaire valide :

Un dictionnaire est un type de données composés de clés/valeurs entourées par des accolades, la structure en est la suivante.

```
        return {
            "neg": sentiment["neg"],
            "neu": sentiment["neu"],
            "pos": sentiment["pos"],
            "compound": sentiment["compound"],
        }
```

Envoyer une requête à l'API et afficher les résultats dans streamlit

On effectue une requête HTTP POST vers l'API FastAPI située à **`http://127.0.0.1:9000/analyse_sentiment/`**, en envoyant le texte saisi par l'utilisateur sous forme de JSON. Il vérifie ensuite que la réponse de l'API ne contient pas de codes d'erreur HTTP (4xx ou 5xx). Si la requête est réussie, il extrait les résultats de l'analyse de sentiment (scores de polarité) de la réponse JSON et les affiche dans l'interface Streamlit.

```
response = requests.post("http://127.0.0.1:9000/analyse_sentiment/",
json={"texte": texte})

# Lève une exception pour les codes d'erreur HTTP (4xx ou 5xx)
response.raise_for_status()

sentiment = response.json()

st.write("Résultats de l'analyse :")
st.write(f"Polarité négative : {sentiment['neg']}")
st.write(f"Polarité neutre : {sentiment['neu']}")
st.write(f"Polarité positive : {sentiment['pos']}")
st.write(f"Score composé : {sentiment['compound']}")
```

Gérer les résultats

On interprète le score de sentiment composé (**`sentiment['compound']`**) renvoyé par VADER pour catégoriser le sentiment global du texte comme positif, négatif ou neutre, et affiche le résultat correspondant dans l'interface Streamlit. La journalisation avec **`loguru`** enregistre également le score de sentiment complet.

```
if sentiment['compound'] >= 0.05 :
      st.write("Sentiment global : Positif □")
elif sentiment['compound'] <= -0.05 :
      st.write("Sentiment global : Négatif □")
else :
      st.write("Sentiment global : Neutre □")
      logger.info(f"Résultats affichés: {sentiment}")
```

Gérer les exceptions :

Les blocs **`try...except`** côté client et serveur garantissent la robustesse de l'application. Le client gère les erreurs de communication avec l'API, informe l'utilisateur et journalise pour le débogage, tandis que le serveur gère les erreurs d'analyse, renvoyant des réponses d'erreur HTTP claires.

du côté streamlit

```
if st.button("Analyser"):
# Si il y a du text alors
    if texte:
        logger.info(f"Texte à analyser: {texte}")
        try:

            # envoie du texte à l'API
            response =
requests.post("http://127.0.0.1:9000/analyse_sentiment/",
json={"texte": texte})

        # traitement de la réponse
        except requests.exceptions.RequestException as e:
            st.error(f"Erreur de connexion à l'API : {e}")
            logger.error(f"Erreur de connexion à l'API : {e}")
        except Exception as e :
            st.error(f"Une erreur est survenue: {e}")
            logger.error(f"Une erreur est survenue: {e}")
    else:
        st.write("Veuillez entrer du texte pour l'analyse.")
```

du côté Fast API

```
@app.post("/analyse_sentiment/")
async def analyse_sentiment(texte_object: Texte):
    logger.info(f"Analyse du texte: {texte_object.texte}")
    try:
        sentiment = sia.polarity_scores(texte_object.texte)
        logger.info(f"Résultats: {sentiment}")

        # on retourne la réponse


    except Exception as e:
        logger.error(f"Erreur lors de l'analyse: {e}")
        raise HTTPException(status_code=500, detail=str(e))
```